

Análisis de métodos exactos y metaheurísticas aplicados al *Graph coloring problem*

Joaquín Andrés Aravena Uribe (*N° 2021454110*), Juan José E. Altamirano Paredes (*N° 2022403951*), Diego Antonio Aravena Díaz (*N° 2020446296*), Martin Antonio Hurtado Echegoyen (*N° 2021418253*), Javiera Karina Montesinos Abarca (*N° 2021442375*), Alan Matias Wilson Germany (*N° 2021448250*)

Departamento de Ingeniería Industrial, Universidad de Concepción
Optimización II, Grupo 15

Resumen

El presente trabajo se enfoca en el problema de *graph coloring*, o simplemente *coloring*. En un principio se describen algunos tipos de este problema, los cuales amplían la comprensión sobre esta área de estudio, para luego enfocarse en el subproblema conocido como *vertex coloring*, o coloreo de vértices, el cual consiste en asignar colores a los vértices de un grafo de tal manera que ningún par de vértices adyacentes compartan el mismo color, utilizando la menor cantidad de colores posible.

El análisis se centra en los aspectos fundamentales del coloreo de vértices. Se plantea un modelo de programación lineal y 2 metaheurísticas, diseñadas para resolver este tipo de problema en grafos de distintos tamaños. Cada uno de los métodos de resolución se implementan con distintas librerías de Python, cuyas soluciones se analizan y comparan para elaborar conclusiones.

1. Introducción

La teoría de grafos es una rama de la matemática discreta de gran utilidad para poder resolver problemas de optimización. Uno de los más populares es el coloreo de grafos.

Para plantear un problema de coloring, es necesario contar con un grafo $G(V, E)$, en donde V es el conjunto de todos los vértices v_i y E es el conjunto de todas las aristas $\{v_i, v_j\}$ tal que $v_i, v_j \in V$.

El objetivo del problema es encontrar la menor cantidad de colores necesarios para cumplir determinadas condiciones, las cuales dependen de cómo se aborde el problema. Puede ser enfocado en el coloreo de sus aristas, vértices o incluso en las caras formadas por grafos planares [Perera and Lanel(2016)], además de nuevas restricciones que se ajusten a la aplicación correspondiente

A continuación se describen brevemente distintos tipos de coloring:

1. **Coloración de vértices** (*Vertex Coloring*): En este tipo de coloración, se asignan colores a los vértices de un grafo de manera que no haya dos vértices adyacentes con el mismo color. El objetivo es minimizar el número de colores utilizados. El número mínimo de colores necesarios para colorear un grafo se llama el número cromático del grafo.
2. **Coloración de aristas** (*Edge Coloring*): En la coloración de aristas, se asignan colores a las aristas de un grafo de manera que no haya dos aristas adyacentes con el mismo color. El número mínimo de colores necesarios para colorear todas las aristas de un grafo se llama el índice cromático del grafo.

3. **Coloración fuerte de vértices** (*Strong Vertex Coloring*): En este tipo de coloración, se asignan colores a los vértices de un grafo de manera que no solo los vértices adyacentes tengan colores diferentes, sino que también los vértices que tienen un vecino en común deben tener colores diferentes.
4. **Coloración de aristas fuerte** (*Strong Edge Coloring*): Similar a la coloración fuerte de vértices, en la coloración fuerte de aristas se asignan colores a las aristas de un grafo de manera que no solo las aristas adyacentes tengan colores diferentes, sino que también las aristas que comparten un vértice en común deben tener colores diferentes.
5. **Coloración total** (*Total Coloring*): En este tipo de coloración, se asignan colores a los vértices de un grafo de manera que los vértices adyacentes y las aristas incidentes a un vértice tengan colores diferentes.

El presente trabajo se enfoca en el *Vertex Coloring Problem (VCP)*, el cual busca minimizar el número de colores utilizados, y, en el mejor de los casos, encontrar el número cromático del grafo.

2. Estado del Arte

En los últimos 10 años se han llevado a cabo múltiples investigaciones relacionadas al *vertex coloring*; muchas de ellas originadas por problemas de *graph coloring*, entre ellas se puede encontrar el trabajo expuesto por [Demange et al.(2015)Demange, Ekim, Ries and Tanasescu]. centrado en aplicaciones para problemas específicos de coloración de grafos.

Dentro de los principales focos de investigación están los modelos de programación lineal (ILP), entre ellos modelos centrados en la programación lineal con restricciones, a través del aporte de [Lewis(2021)], buscando que estos sean lo más sencillos posibles para evitar mayores inconvenientes a la hora de encontrar la optimalidad del problema, como concluyen [Jabrayilov and Mutzel(2018)].

Anteriormente, [Lozin and Malyshev(2017)] llevaron un trabajo similar al estudio sobre el vertex coloring con ciertas restricciones, centrándose en subgrafos inducidos prohibidos con un máximo de 4 vértices. (Si existe un subgrafo prohibido significa que estamos estableciendo una restricción, sobre que subgrafos pueden o no estar presentes en el grafo).

Por otro lado, [Perera and Lanel(2016)] estudiaron una aplicación del vertex coloring en un ámbito académico organizacional, a través de la organización de los cursos de una universidad, con el fin de minimizar el tiempo inactivo, utilizándolo de base para programar las sesiones prácticas para ellos. Paralelamente, [Galinier et al.(2013)Galinier, Hamiez, Hao and Porumbel] incorpora el estudio del vertex coloring al área de la comunicación, en donde utiliza el modelamiento del problema para lograr minimizar la interferencia entre equipos de comunicación geográficamente cercanos.[Dey et al.(2018)Dey, Son, Kumar, Selvachandran and Quek]

Además, se ha hecho énfasis en el desarrollo de nuevos algoritmos heurísticos y metaheurísticos que busquen mejorar el desempeño computacional, como lo estudia [Chahkandi and Mirzaei(2015)] a través del algoritmo ABC, una solución para comprender el comportamiento de las abejas en sus colonias. [Marappan and Sethumadhavan(2013)] a su vez ha dado lugar en avances para la comprensión de algoritmos genéticos, como también [Said et al.(2016)Said, Talea, Bakali and Smarandache] realizó estudios con enfoques un poco alejados de lo habitual, como problemas neutrosóficos dinámicos y de valor único, utilizados en situaciones de la vida real.

3. Notación general y datos utilizados

Dado un grafo $G(V, E)$, definimos como número cromático de G a la menor cantidad de colores utilizada para colorear los vértices de un grafo tal que los vértices adyacentes no compartan el mismo color. Se denota de la siguiente manera: $\chi(G)$.

Además, se crea el conjunto J de colores que se podrían utilizar para colorear el grafo. Notemos que si un gráfico tiene n vértices, entonces $1 \leq \chi(G) \leq n$, es decir, el número cromático está entre 1, para un grafo con 1 vértice, y n para un grafo completo, en donde todos los vértices son adyacentes entre sí, por lo que se necesitaría un color para cada vértice.

Lo anterior contribuye a establecer un supremo a la cantidad de colores del conjunto J , lo cual facilita la tarea de modelarlo en un software computacional, definiendo que para cualquier grafo, el conjunto J tendrá la misma cantidad de elementos que el conjunto V de vértices del grafo G .

Los metodos de resolución se aplican a 10 grafos diferentes recolectados de la base de datos OR-Library, la cual nos entrega instancias con el objetivo de ser resueltas como un *vertex coloring problem*, específicamente los llamados gcol1, gcol2, ..., gcol10. Es necesario mencionar que las soluciones óptimas de los problemas recién mencionados no son conocidas, por lo tanto, no es posible evaluar el desempeño de los métodos considerando la solución óptima.

4. Método exacto

El método en cuestión, expuesto por [Lewis(2021)], es un modelo de programación lineal entera, debido a que se utilizan variables binarias para poder plantearlo.

Para trabajar numéricamente con colores, el conjunto J estará representado por números enteros entre 1 y n , con n el número de vértices del grafo.

Parámetros:

$G(V, E)$: Grafo con su respectivo conjunto de vértices y aristas.

J : Conjunto de colores que se podrían utilizar.

Variables de decisión:

X_{ij} : Variable binaria. 1 si el vértice v_i es coloreado con el color j . 0 en otro caso.

Y_j : Variable binaria. 1 si el color j es utilizado en al menos 1 vértice. 0 en otro caso.

Función objetivo:

$$\text{Minimizar } \sum_{j=1}^n Y_j$$

Para plantear la función objetivo, se establece una sumatoria de la función binaria que determina si un color ha sido utilizado al menos una vez. De esta forma se garantiza encontrar el número cromático más bajo.

Sujeto a:

$$X_{ij} + X_{lj} \leq Y_j; \quad \forall \{v_i, v_l\} \in E, \quad \forall j \in \{1, \dots, n\} \quad (1)$$

$$\sum_{j=1}^n X_{ij} = 1; \quad \forall v_i \in V \quad (2)$$

$$X_{ij} \in \{0, 1\}; \quad \forall v_i \in V, \quad \forall j \in \{1, \dots, n\} \quad (3)$$

$$Y_j \in \{0, 1\}; \quad \forall j \in \{1, \dots, n\} \quad (4)$$

Estas restricciones se pueden entender de la siguiente manera:

1. Asegura que 2 vértices adyacentes no pueden utilizar el mismo color. Es decir, en caso de que el color j se utilice en un vértice ($Y_j = 1$), la suma entre las variables X de dos vértices adyacentes debe ser a lo más 1, indicando que a lo más un vértice debe utilizar el color j .
2. Esta restricción indica que a cada vértice se le debe asignar exactamente un color.
3. Determina la naturaleza de la variable de decisión, especificando que es una variable binaria.
4. Al igual que la restricción anterior, establece la naturaleza binaria de la variable.

4.1. Implementación

Una vez definido el modelo, se implementa en Python con los paquetes DOCPlex y Gurobi de manera independiente, aplicandolo a los 10 grafos mencionados anteriormente.

Además de esto, se hace uso de las bibliotecas matplotlib.pyplot, networkx, matplotlib.colors y random para la visualización del grafo, sin embargo no son necesarias para la resolución del problema, ya que esta es algebraica.

Es necesario destacar que la forma de representar los grafos en la base de datos es diferente a la utilizada en el código, por lo tanto se tuvo que diseñar un algoritmo adicional para adaptarlo.

Cabe destacar que debido a la complejidad NP del problema y la dimensión de los grafos, este tiene un muy alto costo computacional [Lewis(2021)], por lo que el modelo no es capaz de devolver la solución óptima en un tiempo razonable, es por eso que se establecieron 3 límites de tiempo para procesar el modelo: 1 minuto, 20 minutos y 1 hora. La distribución de tiempos y descripción cuantitativa de cada grafo se expone a continuación:

Tabla 1: Información de 10 Grafos (Fuente: Elaboración propia).

Grafo	Vértices	Aristas	Tiempo (s)
<i>Grafo 1</i>	100	2487	1200
<i>Grafo 2</i>	100	2487	3600
<i>Grafo 3</i>	100	2482	60
<i>Grafo 4</i>	100	2503	1200
<i>Grafo 5</i>	100	2450	1200
<i>Grafo 6</i>	100	2537	1200
<i>Grafo 7</i>	100	2505	60
<i>Grafo 8</i>	100	2479	3600
<i>Grafo 9</i>	100	2486	60
<i>Grafo 10</i>	100	2506	1200

4.2. Resolución

Se resuelven los 10 grafos propuestos en la base de datos. Como ejemplo de visualización, se presentan los resultados obtenidos por el grafo 1.

Para comenzar, se presenta el grafo sin colorear:

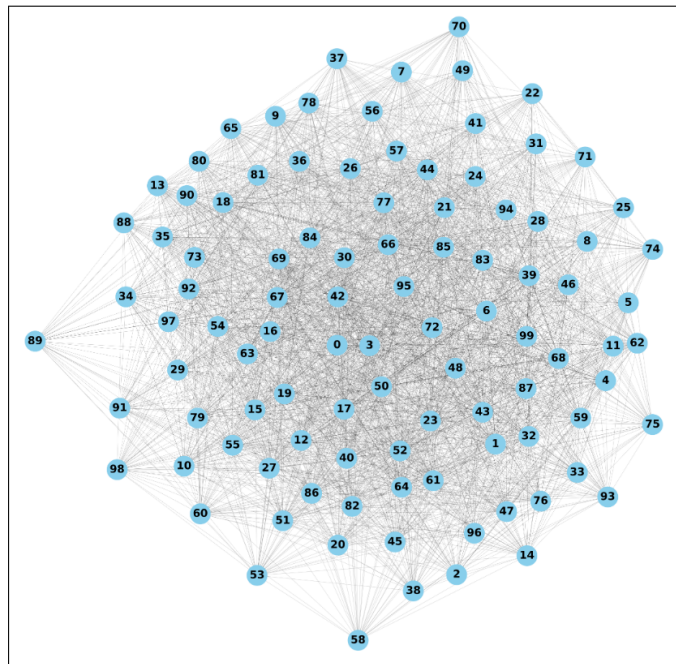


Figura 1: Grafo 1 sin colorear (Fuente: Elaboración Propia, Python).

Resultados de Grafo 1 en DOCplex:

Solución factible obtenida: 18 colores

Los resultados de la variable X_{ij} se agrupan en la siguiente matriz:

$$\begin{bmatrix} X_{0,78} & X_{1,78} & X_{2,56} & X_{3,34} & X_{4,29} & X_{5,97} & X_{6,45} & X_{7,69} & X_{8,8} & X_{9,54} \\ X_{10,56} & X_{11,56} & X_{12,73} & X_{13,1} & X_{14,73} & X_{15,11} & X_{16,85} & X_{17,34} & X_{18,73} & X_{19,73} \\ X_{20,33} & X_{21,11} & X_{22,69} & X_{23,69} & X_{24,51} & X_{25,49} & X_{26,34} & X_{27,69} & X_{28,51} & X_{29,19} \\ X_{30,19} & X_{31,45} & X_{32,11} & X_{33,56} & X_{34,49} & X_{35,51} & X_{36,33} & X_{37,19} & X_{38,45} & X_{39,54} \\ X_{40,45} & X_{41,19} & X_{42,25} & X_{43,69} & X_{44,8} & X_{45,49} & X_{46,34} & X_{47,11} & X_{48,85} & X_{49,69} \\ X_{50,73} & X_{51,8} & X_{52,1} & X_{53,54} & X_{54,11} & X_{55,19} & X_{56,49} & X_{57,45} & X_{58,19} & X_{59,49} \\ X_{60,45} & X_{61,51} & X_{62,54} & X_{63,78} & X_{64,1} & X_{65,1} & X_{66,1} & X_{67,25} & X_{68,56} & X_{69,8} \\ X_{70,11} & X_{71,85} & X_{72,78} & X_{73,69} & X_{74,25} & X_{75,11} & X_{76,54} & X_{77,49} & X_{78,56} & X_{79,69} \\ X_{80,29} & X_{81,85} & X_{82,33} & X_{83,33} & X_{84,29} & X_{85,73} & X_{86,51} & X_{87,8} & X_{88,25} & X_{89,51} \\ X_{90,73} & X_{91,78} & X_{92,34} & X_{93,85} & X_{94,56} & X_{95,34} & X_{96,56} & X_{97,8} & X_{98,78} & X_{99,97} \end{bmatrix} = O$$

Donde O es una matriz cuadrada de unos con $\dim(O) = 10$.

Los resultados de la variable Y_j se agrupan en el siguiente vector:

$$Y = \begin{bmatrix} Y_1 & Y_8 & Y_{11} & Y_{19} & Y_{25} & Y_{29} & Y_{33} & Y_{34} & Y_{45} & Y_{49} \\ Y_{51} & Y_{54} & Y_{56} & Y_{69} & Y_{73} & Y_{78} & Y_{85} & Y_{97} \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & & \end{bmatrix}$$

Los resultados anteriores determinan el color asignado a cada vector, lo cual puede ser agrupado en las siguientes tablas:

Tabla 2: Asiganción de colores con DOCplex (Dividido) (Fuente: Elaboración propia).

Vértices	Color
13, 52, 64, 65, 66	
8, 44, 51, 69, 87, 97	
15, 21, 32, 47, 54, 70, 75	
29, 30, 37, 41, 55, 58	
42, 67, 74, 88	
4, 80, 84	
20, 36, 82, 83	
3, 17, 26, 46, 92, 95	
6, 31, 38, 40, 57, 60	

(a) Colores Asignados a Vértices de Grafo 1

Vértices	Color
25, 34, 45, 56, 59, 77	
24, 28, 35, 61, 86, 89	
9, 39, 53, 62, 76	
2, 10, 11, 33, 68, 78, 94, 96	
7, 22, 23, 27, 43, 49, 73, 79	
12, 14, 18, 19, 50, 85, 90	
0, 1, 63, 72, 91, 98	
16, 48, 71, 81, 93	
5, 99	

(b) Colores Asignados a Vértices de Grafo 1

De esta manera, el coloreo resultante del grafo 1 es el siguiente:

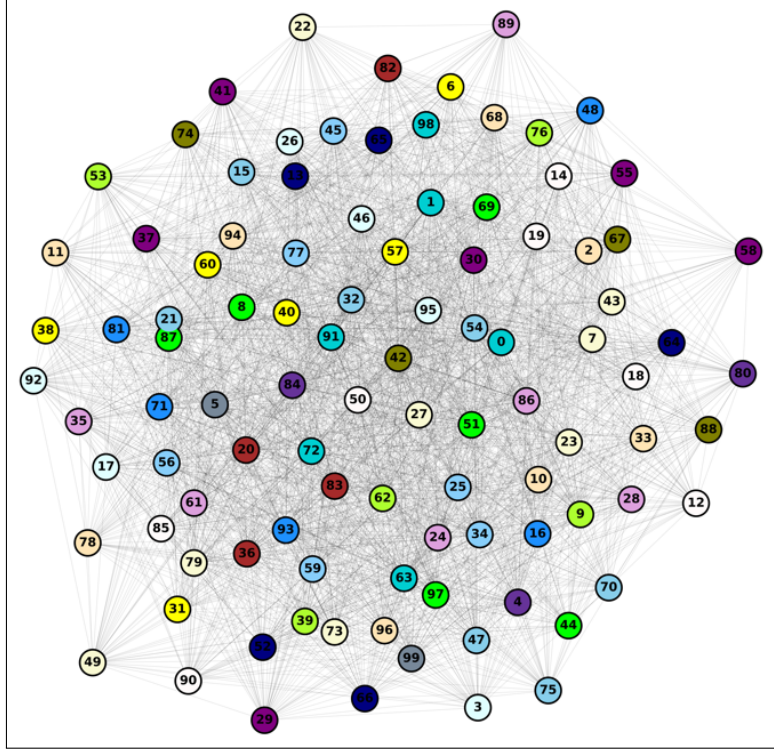


Figura 2: Grafo 1 coloreado DOCplex (Fuente: Elaboración Propia, Python).

Resultados de Grafo 1 en Gurobi

Solución factible obtenida: 21 colores.

Los resultados de la variable X_{ij} se agrupan en la siguiente matriz:

$$\begin{bmatrix} X_{(0,49)} & X_{(1,49)} & X_{(2,99)} & X_{(3,49)} & X_{(4,98)} & X_{(5,99)} & X_{(6,98)} & X_{(7,49)} & X_{(8,97)} & X_{(9,96)} \\ X_{(10,95)} & X_{(11,98)} & X_{(12,99)} & X_{(13,98)} & X_{(14,97)} & X_{(15,94)} & X_{(16,96)} & X_{(17,95)} & X_{(18,97)} & X_{(19,97)} \\ X_{(20,99)} & X_{(21,94)} & X_{(22,95)} & X_{(23,93)} & X_{(24,92)} & X_{(25,94)} & X_{(26,93)} & X_{(27,91)} & X_{(28,92)} & X_{(29,98)} \\ X_{(30,91)} & X_{(31,90)} & X_{(32,97)} & X_{(33,93)} & X_{(34,94)} & X_{(35,96)} & X_{(36,90)} & X_{(37,99)} & X_{(38,90)} & X_{(39,89)} \\ X_{(40,92)} & X_{(41,91)} & X_{(42,88)} & X_{(43,91)} & X_{(44,89)} & X_{(45,96)} & X_{(46,87)} & X_{(47,89)} & X_{(48,93)} & X_{(49,95)} \\ X_{(50,99)} & X_{(51,86)} & X_{(52,88)} & X_{(53,89)} & X_{(54,87)} & X_{(55,85)} & X_{(56,87)} & X_{(57,89)} & X_{(58,98)} & X_{(59,87)} \\ X_{(60,49)} & X_{(61,93)} & X_{(62,85)} & X_{(63,88)} & X_{(64,94)} & X_{(65,86)} & X_{(66,49)} & X_{(67,90)} & X_{(68,95)} & X_{(69,90)} \\ X_{(70,99)} & X_{(71,96)} & X_{(72,93)} & X_{(73,84)} & X_{(74,94)} & X_{(75,90)} & X_{(76,86)} & X_{(77,92)} & X_{(78,83)} & X_{(79,84)} \\ X_{(80,85)} & X_{(81,85)} & X_{(82,96)} & X_{(83,82)} & X_{(84,85)} & X_{(85,81)} & X_{(86,83)} & X_{(87,80)} & X_{(88,95)} & X_{(89,93)} \\ X_{(90,85)} & X_{(91,88)} & X_{(92,81)} & X_{(93,88)} & X_{(94,82)} & X_{(95,97)} & X_{(96,80)} & X_{(97,85)} & X_{(98,81)} & X_{(99,83)} \end{bmatrix} = O$$

Donde O es una matriz cuadrada de unos, donde $\dim(O) = 10$.

Los resultados de la variable Y_j se agrupan en el siguiente vector:

$$Y = \begin{bmatrix} Y_{49} & Y_{80} & Y_{81} & Y_{82} & Y_{83} & Y_{84} & Y_{85} & Y_{86} & Y_{87} & Y_{88} \\ Y_{89} & Y_{90} & Y_{91} & Y_{92} & Y_{93} & Y_{94} & Y_{95} & Y_{96} & Y_{97} & Y_{98} \\ Y_{99} \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 \end{bmatrix}$$

Los resultados anteriores determinan el color asignado a cada vector, lo cual puede ser agrupado en las siguientes tablas: De esta manera, el coloreo resultante del grafo 1 es el siguiente:

Tabla 3: Asignación de colores con Gurobi (Dividido) (Fuente: Elaboración Propia).

Vértices	Color
0, 1, 3, 7, 60, 66	
87, 96	
85, 92, 98	
83, 94	
78, 86, 99	
73, 79	
55, 62, 80, 81, 84, 90, 97	
51, 65, 76	
46, 54, 56, 59	
42, 52, 63, 91, 93	
39, 44, 47, 53, 57	

(a) Colores Asignados a Vértices de Grafo 1

Vértices	Color
31, 36, 38, 67, 69, 75	
27, 30, 41, 43	
24, 28, 40, 77	
23, 26, 33, 48, 61, 72, 89	
15, 21, 25, 34, 64, 74	
10, 17, 22, 49, 68, 88	
9, 16, 35, 45, 71, 82	
8, 14, 18, 19, 32, 95	
4, 6, 11, 13, 29, 58	
2, 5, 12, 20, 37, 50, 70	

(b) Colores Asignados a Vértices de Grafo 1

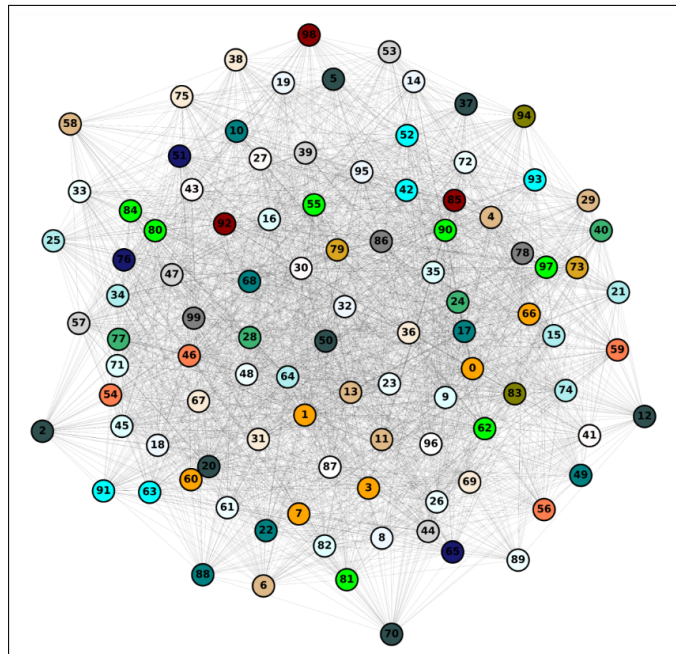


Figura 3: Grafo 1 coloreado Gurobi (Fuente: Elaboración Propia, Python).

4.3. Comparación de resultados:

A continuación se dividen los grafos según el tiempo límite designado para resolver su respectivo coloreo. Los siguientes gráficos muestran la comparación entre soluciones obtenidas por ambos solver para cada uno de los casos:

- Tiempo límite de 1 minuto

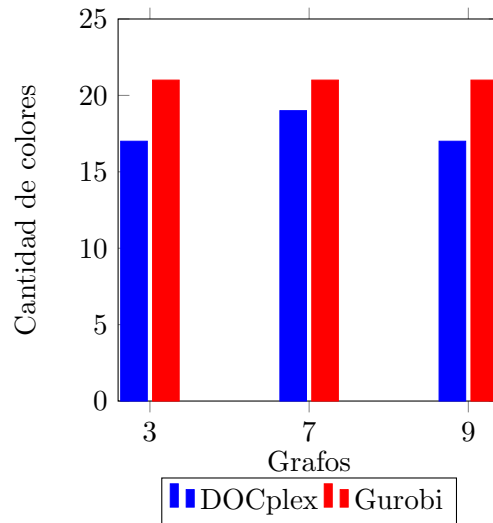


Figura 4: Comparación de resultados en pruebas de 1 minuto (Fuente: Elaboración Propia, Python).

- Tiempo límite de 20 minutos

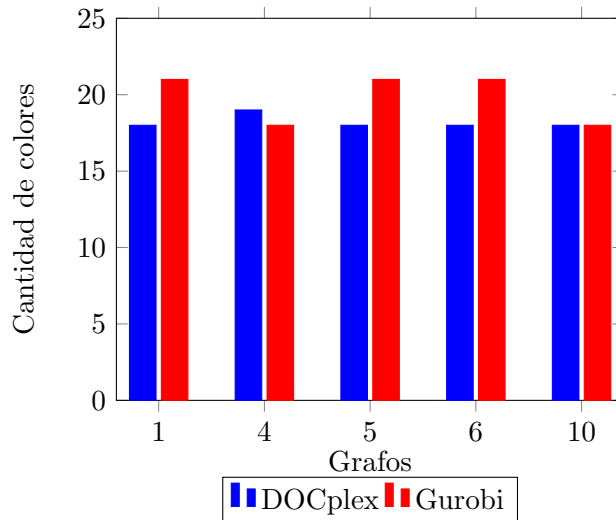


Figura 5: Comparación de resultados en pruebas de 20 minutos (Fuente: Elaboración Propia, Python).

- Tiempo límite de 1 hora

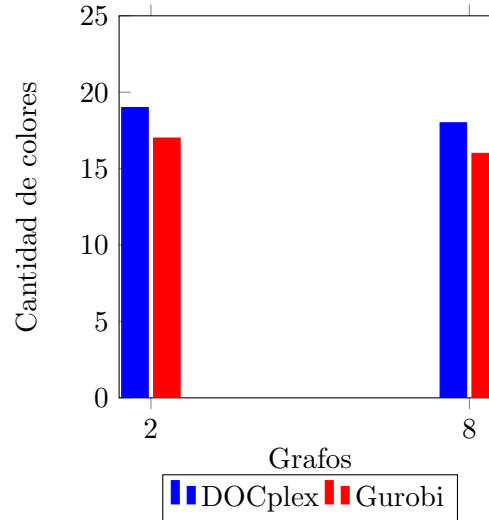


Figura 6: Comparación de resultados en pruebas de 1 hora (Fuente: Elaboración Propia, Python).

Analizando los resultados del método exacto, es posible notar que en implementaciones de poco tiempo (1 minuto) DOCplex entrega mejores resultados que Gurobi. Avanzando a los 20 minutos, esta tendencia se mantiene, aunque Gurobi mejora sus soluciones, para que finalmente, con un tiempo de procesamiento de 1 hora, Gurobi obtenga mejores resultados que DOCplex

5. Metaheurísticas

Los algoritmos metaheurísticos son técnicas de optimización que utilizan los algoritmos heurísticos como base fundamental. Las heurísticas son algoritmos de optimización que buscan soluciones a problemas mediante ensayo y error, sin garantizar la óptima global, pero proporcionando soluciones factibles y si el algoritmo posee un diseño adecuado, soluciones de calidad. Por su parte, los algoritmos metaheurísticos representan un conjunto de técnicas empleadas en la resolución de problemas de optimización combinatoria, especialmente cuando los métodos heurísticos convencionales no son eficientes. Estos enfoques adaptan, guían y modifican algoritmos heurísticos para explorar y explotar un espacio de búsqueda de manera más efectiva [Boussaïd et al.(2013)Boussaïd, Lepagnot and Siarry]. A continuación se presentan dos metaheurísticas que resuelven el problema en cuestión:

5.1. Búsqueda Local Iterada (ILS):

La Búsqueda local iterada es un enfoque de búsqueda heurística en la optimización combinatoria que opera de manera iterativa y cíclica para mejorar soluciones. Comienza con una solución inicial y utiliza una búsqueda local para encontrar una solución cercana. Luego, introduce perturbaciones en la solución para escapar de posibles óptimos locales, finalmente, repite el proceso para optimizar continuamente las soluciones, en donde, progresivamente se acerca a soluciones de mayor calidad al aumentar las iteraciones, con el riesgo de quedarse estancado en largos periodos tiempos.

El pseudocódigo del algoritmo se presenta a continuación:

Algorithm 1: Búsqueda Local Iterada

```
1  $s_0 = \text{GenerarSoluciónInicial}()$  ;  
2  $s = \text{BúsquedaLocal}(s_0)$ ;  
3 while Condición de tiempo no es alcanzada do  
4    $s' = \text{Perturbación}(s)$ ;  
5    $s'' = \text{BúsquedaLocal}(s')$ ;  
6    $s = \text{CriterioDeAceptación}(s, s'')$   
7 end
```

Este algoritmo comienza generando una solución inicial de manera aleatoria, la cual consiste en un vector de tamaño n (equivalente al número de vértices del grafo), en donde cada entrada contiene un número que representa el color que se le asigna al vértice de dicha posición. Es necesario destacar, que esta solución no necesariamente es factible.

Luego, a esta solución se le aplica una búsqueda local, la cual se mueve entre soluciones vecinas, intentando reducir el número de conflictos. Un par de vértices están en conflicto si ambos tienen asignado el mismo color. La búsqueda local intenta iterativamente reparar la asignación de colores actual, guiada por una función de evaluación (*"fitness"*) que cuenta el número total de conflictos [Chiarandini and Stützle(2007)]. De esta manera, la búsqueda local retorna una solución propuesta sin conflictos, es decir, un coloreo de vértices factible utilizando la misma cantidad de colores que la solución infactible inicial.

Por otro lado, se define una condición de término, en donde se establece un tiempo máximo de operación del algoritmo. Posteriormente, la solución propuesta ingresa a un ciclo, el cual itera hasta que la condición de término es alcanzada. Dentro del ciclo, se realiza la siguiente perturbación a la solución propuesta:

1. Se escoge al azar un color utilizado en la solución propuesta.
2. Se elimina el color seleccionado de la solución propuesta.
3. Se reasigna un color distinto (del conjunto de colores utilizados) a cada uno de los vértices que poseían el color eliminado.
4. Se obtiene una solución perturbada.

Cabe destacar que la solución perturbada por lo general es infactible, debido a que se crean conflictos por la asignación aleatoria de colores.

Es aquí cuando se aplica una nueva búsqueda local a la solución perturbada. En caso de que la búsqueda local no consiga reducir los conflictos a 0 en un tiempo de 1 segundo, la búsqueda se detiene y retorna una solución con el menor número de conflictos que obtuvo.

Se define este límite de tiempo en la búsqueda local de manera experimental, luego de notar que el tiempo promedio que tarda esta heurística en factibilizar una solución es considerablemente inferior a un segundo. Una vez alcanzado el tiempo de 1 segundo, la búsqueda no conseguiría una solución factible y sería necesario realizar una nueva perturbación.

Finalmente, se establece un criterio de aceptación que selecciona la nueva solución propuesta. El criterio es el siguiente:

Si el número de colores utilizados en s'' es menor que el de s , y s'' es factible, entonces, se actualiza la nueva solución propuesta a s'' .

El algoritmo termina cuando se alcance el tiempo máximo establecido previamente.

5.2. Algoritmo Genético (AG):

Se inspira en la evolución y selección natural para encontrar soluciones factibles a problemas complejo. Al igual que ILS, el Algoritmo Genético es un enfoque de búsqueda metaheurística en la optimización combinatoria. Esta comienza con una población inicial de posibles soluciones llamadas cromosomas y utiliza una función de aptitud (*“fitness”*) para evaluar su calidad. Luego, selecciona los cromosomas más aptos para la reproducción y combina información genética a través del cruzamiento. Además, introduce mutaciones aleatorias para diversificar la población y evitar óptimos locales. A lo largo de múltiples generaciones, el AG repite este proceso, lo que gradualmente se encamina hacia soluciones de mayor calidad a medida que transcurren las generaciones.

El pseudocódigo del algoritmo se presenta a continuación:

Algorithm 2: Algoritmo Genético

```
1 poblacion = generación aleatoria de cromosomas;
2 mejor solucion = solucion inicial;
3 while condición de término no es alcanzada do
4     padres = seleccion();
5     hijo = crossover(padres);
6     hijo = mutacion(hijo);
7     poblacion.añadir(hijo);
8     mejor solucion = evaluar(hijo);
9 end
```

Se inicia el código con la creación de una población, es decir, un conjunto de cromosomas (posibles soluciones), donde este grupo tiene las características de, primero, no necesariamente estar compuesto por soluciones factibles y segundo, tener un largo de 50 elementos, número determinado experimentalmente al ser lo suficientemente eficiente para el desempeño del algoritmo [Hindi and Yampolskiy(2012)]. Cada cromosoma es un vector de 100 elementos, en donde cada entrada contiene un número que representa el color que se le asigna al vértice de dicha posición.

Además, se inicializa una solución inicial factible con la que se compararán las obtenidas durante el proceso, que se irá actualizando cuando disminuya el número de colores utilizados. Esta solución se obtendrá con una única búsqueda local.

Posteriormente a la población se le aplica una función de selección que analiza los cromosomas en busca de los más aptos. Luego, los cruza entre ellos para obtener un cromosoma descendiente (hijo). Este, se obtiene al hacer un corte aleatorio en ambos padres, y juntar las mitades. A este nuevo cromosoma hijo se le aplica una función de mutación, la cual cambia el color de uno o más nodos en conflicto por otro disponible. La mutación, ocurre bajo una cierta probabilidad establecida de 0.7, la cual fue determinada experimentalmente [Hindi and Yampolskiy(2012)]. Es en este proceso, donde ocurre la reducción de colores, sin embargo esto no ocurre todas las veces, en la que en el peor de los casos, la cantidad de colores se mantendrá. Tanto las funciones de mutación como de selección, tienen dos variantes, las cuales, dependiendo de la calidad de las soluciones, se le asigna el proceso 1 o 2. Si el cromosoma con menor cantidad de conflictos de la población posee una puntuación mayor a 4 (mala), se ocupan las primeras variantes, dado que estas aplican modificaciones más drásticas a los cromosomas. En cambio, si se encuentra bajo este valor, se realiza el proceso 2, con cambios más sutiles.

El método que utiliza el algoritmo para verificar qué cromosoma es más calificado o no, es la función fitness, función que por cada cromosoma analiza la cantidad de conflictos que tiene, siendo

un conflicto, cuando dos vértices adyacentes comparten un mismo color. Por cada conflicto, se suma 1 a la puntuación de la solución. Mientras menos conflictos tenga la solución, mejor es esta, lográndose la factibilidad cuando la puntuación llega a 0.

Todo este proceso se realiza de manera iterativa hasta que se cumple una condición determinada, que puede ser un tiempo límite de procesos o una cantidad determinada de iteraciones.

5.3. Implementación:

Para evaluar el rendimiento de cada metaheurística, se implementan los algoritmos en lenguaje Python en cada una de las instancias requeridas, las especificaciones de detallan en los siguientes casos de estudio:

- C1) Una instancia extra proveniente de la base de datos OR-Library con un número cromático conocido, estableciendo un tiempo máximo de 120 segundos.
- C2) Instancia 1 (gc01), estableciendo un tiempo máximo de 300 segundos.
- C3) 10 instancias definidas en la sección 3, estableciendo un tiempo máximo de 60 segundos.

Es importante mencionar que para cada instancia del caso 3, se utilizan 5 semillas distintas, y se rescata la mejor solución obtenida en estas 5 iteraciones.

Además, para los casos 2 y 3, adicionalmente se resolverá mediante Gurobi y Python con el fin de realizar una comparación de resultados entre métodos exactos y metaheurísticas.

En la implementación de cada algoritmo, se emplean los paquetes de Python, *numpy* y *time*, con el fin de satisfacer los requisitos específicos del diseño de cada uno de ellos. Además, en el caso 2, se utiliza el paquete "matplotlib.pyplot" para graficar resultados.

5.4. Comparación de resultados:

Caso de estudio 1:

Como se menciona anteriormente, las instancias definidas en la sección 3, no poseen un óptimo conocido. Es por esto que, para verificar si las metaheurísticas propuestas son capaces de hallar un óptimo global, se trabaja con una instancia extra "queen5_5", grafo de 25 vértices y 160 aristas cuyo óptimo conocido es $\chi(G) = 5$.

Se aplica el algoritmo genético y de búsqueda local iterada (ILS) y se obtienen los siguientes resultados:

Tabla 4: Resultados de caso de estudio 1. (Fuente: Elaboración propia).

Algoritmo	Solución obtenida	Tiempo (s)
ILS	5	4.06
Genético	5	73.347

Lo cual comprueba que las metaheurísticas diseñadas son capaces de alcanzar soluciones óptimas.

Caso de estudio 2:

Para poder comparar gráficamente los resultados de los métodos exactos con los de las metaheurísticas, se resuelve la instancia 1 (gcol1) con Python, Gurobi y cada una de las metaheurísticas, obteniendo los siguientes resultados:

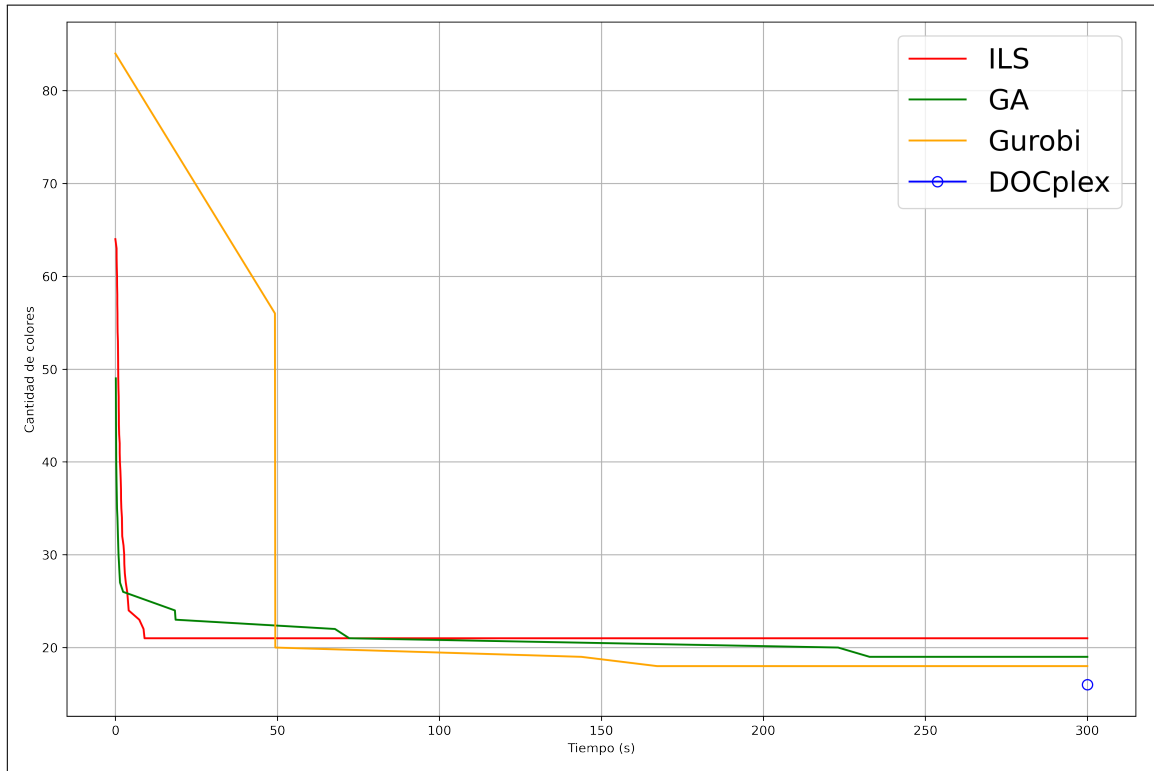


Figura 7: Comparación de resultados caso de estudio 2. (Fuente: Elaboración Propia, Python).

En el gráfico presentado, es posible notar la evolución de las soluciones obtenidas por cada uno de los métodos de resolución. Es importante mencionar que DOCplex no entrega la evolución de los colores obtenidos a lo largo del tiempo, solo retorna el resultado obtenido una vez alcanzado el límite de tiempo, lo cual está representado en el gráfico con un punto en el segundo 300.

Analizando los resultados obtenidos, el algoritmo de búsqueda local iterada (ILS) mejora su solución en un muy corto período de tiempo y de manera brusca. Por otro lado, el algoritmo genético (GA), sigue un comportamiento similar, sin embargo, en los primeros segundos obtiene soluciones inferiores a la otra metaheurística, pero, a diferencia de ILS, en lugar de estancarse, mejora su solución lentamente en el tiempo restante.

Por otro lado, el modelo exacto implementado en Gurobi, tarda un tiempo considerable en encontrar una solución aceptable, atravesando cambios bruscos los cuales se estabilizan luego de 50 segundos, para finalmente llegar a una buena solución.

Caso de estudio 3:

A continuación se tabulan los resultados obtenidos en las 10 instancias definidas en la sección 3, con un tiempo de ejecución de 60 segundos:

Tabla 5: Resultados de caso de estudio 3. (Fuente: Elaboración propia).

Instancia	Gurobi	Solución DOCplex	ILS	AG
<i>gcol1</i>	18	18	19	21
<i>gcol2</i>	19	18	21	21
<i>gcol3</i>	19	17	20	22
<i>gcol4</i>	21	19	20	19
<i>gcol5</i>	22	18	19	22
<i>gcol6</i>	21	18	20	21
<i>gcol7</i>	21	19	20	22
<i>gcol8</i>	20	18	20	23
<i>gcol9</i>	20	17	20	23
<i>gcol10</i>	20	18	20	22

Como podemos ver, el modelo resuelto con DOCplex tiene el mejor desempeño en tiempos acotados, obteniendo la mejor solución en prácticamente todos los grafos. En segundo lugar se encuentra el algoritmo ILS, que en general obtiene un desempeño constante y aceptable. Finalmente están Gurobi y AG. Ambos son, en menor medida, menos aceptables que los métodos anteriores.

Algo a destacar es que las metaheurísticas son constantes con sus soluciones, siguiendo la tendencia de los métodos exactos, evitando una alta variabilidad. Además, si se hace una comparación entre algoritmos aproximados, el claro ganador es ILS, que es incluso superior a un método exacto con el mismo límite de tiempo establecido. Los resultados regulares del algoritmo genético se podrían deber a que el diseño del programa en Python no está optimizado, lo cual aumentaría su costo computacional y tendría repercusiones en su desempeño.

6. Conclusiones

Los resultados de los modelos exactos, si bien tienen diferencias bajas, cuando se trata de un problema real de industrias u otras aplicaciones, utilizar “un color” más puede conllevar grandes costos adicionales, por lo que es relevante saber qué solver tiene un mejor desempeño.

Por otro lado, analizando los resultados de las metaheurísticas, se puede notar que ambos algoritmos obtienen buenos resultados en muy pocos segundos, destacando que, en general, la búsqueda local iterada obtiene mejores soluciones y en menores tiempos que el Algoritmo Genético, sin embargo, a medida que se aumenta el tiempo de operación de ambos algoritmos, tienen soluciones similares. Además, se comprobó que ambas metaheurísticas son capaces de obtener soluciones óptimas de instancias con un número cromático conocido.

En conclusión, para la resolución del coloreo de vértices, las metaheurísticas son ideales para resolver instancias de gran tamaño en muy cortos periodos de tiempo (segundos). Esto podría ser especialmente útil en problemas reales donde se necesiten resolver una gran cantidad de instancias en un tiempo muy acotado.

Referencias

- [Boussaïd et al.(2013)Boussaïd, Lepagnot and Siarry] Boussaïd, I., Lepagnot, J., Siarry, P., 2013. A survey on optimization metaheuristics. *Information Sciences* 237, 82–117. Prediction, Control and Diagnosis using Advanced Neural Computations.
- [Chahkandi and Mirzaei(2015)] Chahkandi, V., Mirzaei, O., 2015. Vertex coloring based on artificial bee colony algorithm, in: 2015 International Congress on Technology, Communication and Knowledge (ICTCK), pp. 312–317.
- [Chiarandini and Stützle(2007)] Chiarandini, M., Stützle, T., 2007. An application of iterated local search to the graph coloring problem. URL: <https://api.semanticscholar.org/CorpusID:15629864>.
- [Demange et al.(2015)Demange, Ekim, Ries and Tanasescu] Demange, M., Ekim, T., Ries, B., Tanasescu, C., 2015. On some applications of the selective graph coloring problem. *European Journal of Operational Research* 240, 307–314.
- [Dey et al.(2018)Dey, Son, Kumar, Selvachandran and Quek] Dey, A., Son, L.H., Kumar, P.K.K., Selvachandran, G., Quek, S.G., 2018. New concepts on vertex and edge coloring of simple vague graphs. *Symmetry* 10.
- [Galinier et al.(2013)Galinier, Hamiez, Hao and Porumbel] Galinier, P., Hamiez, J.P., Hao, J.K., Porumbel, D., 2013. Recent Advances in Graph Vertex Coloring. volume 38. pp. 505–528.
- [Hindi and Yampolskiy(2012)] Hindi, M., Yampolskiy, R., 2012. Genetic algorithm applied to the graph coloring problem. *Midwest Artificial Intelligence and Cognitive Science Conference* , 60.
- [Jabrayilov and Mutzel(2018)] Jabrayilov, A., Mutzel, P., 2018. New Integer Linear Programming Models for the Vertex Coloring Problem. pp. 640–652.
- [Lewis(2021)] Lewis, R.M.R., 2021. *Guide to Graph Colouring: Algorithms and Applications*. Springer.
- [Lozin and Malyshev(2017)] Lozin, V., Malyshev, D., 2017. Vertex coloring of graphs with few obstructions. *Discrete Applied Mathematics* 216, 273–280.
- [Marappan and Sethumadhavan(2013)] Marappan, R., Sethumadhavan, G., 2013. A new genetic algorithm for graph coloring, in: 2013 Fifth International Conference on Computational Intelligence, Modelling and Simulation, pp. 49–54.
- [Perera and Lanel(2016)] Perera, T., Lanel, G., 2016. A model to optimize university course timetable using graph coloring and integer linear programming. *IOSR Journal of Mathematics* 12, 13–18.
- [Said et al.(2016)Said, Talea, Bakali and Smarandache] Said, B., Talea, M., Bakali, A., Smarandache, F., 2016. Single valued neutrosophic graphs .